DressMe

Dossier de presentation technique

8 agents LangGraph, dont un orchestrateur conversationnel

4 services FastAPI, 1 RAG cite, protocole A2A 0.3.0

Architecture, LLM, entrees / sorties, graphes, algorithmes, limites

---

Architecture, agents, LLM, graphes et algorithmes

Projet de fin d'etudes - React 19, FastAPI, LangGraph, OpenAI, Supabase

1. Resume executif

DressMe est une application de garde-robe intelligente. L'utilisateur photographie ses vetements, l'application les catalogue automatiquement (detecteur YOLO + GPT-4o vision), puis huit agents LangGraph l'aident a s'habiller, a construire des tenues, a preparer une valise, a reperer les lacunes de sa garde-robe et a decider d'un achat. Un orchestrateur unique donne un seul point d'entree conversationnel.

Le principe d'architecture est constant : **le LLM decide et redige, le code deterministe verifie et calcule**. Aucun agent ne peut citer une piece absente du placard, inventer une prevision meteo ou citer une source qui n'existe pas dans le RAG.

Indicateur	Valeur
Agents LangGraph	8 (dont 1 orchestrateur) et 2 orchestrateurs alternatifs
Services FastAPI	BO1 8000, BO3 8001, Help Me Pack 8020, RAG 8010, orchestre 8070
Modeles LLM	GPT-4o, gpt-4o-mini, Qwen2.5-3B + LoRA, Ollama qwen2.5:3b
Corpus RAG	38 PDF, 14 574 chunks, all-MiniLM-L6-v2 (384 dimensions)
Qualite RAG	Hit@5 91,67 %, MRR 85,42 %, precision du contexte 71,67 %
Tests	101 orchestrateur, 85 backend, 30 frontend, 1 095 BO4 (25 ignores)
Protocole agent a agent	A2A 0.3.0 sur JSON-RPC 2.0
Categories de vetements	20 classes detectees et cataloguees
Outils du Trip Agent	10 outils valides par Pydantic + solveur OR-Tools CP-SAT

Ce qui distingue le projet

- **Ancrage fort** : les identifiants de pieces viennent toujours de la vraie garde-robe ; la meteo vient d'Open-Meteo ; les conseils de mode viennent de livres, avec des marqueurs [S#].
- **Verification par code** : une tenue est validee par des regles (robe seule ou haut + bas, chaussures obligatoires, pas deux pieces du meme role), un verdict d'achat est valide par du code.
- **Degradation douce** : RAG, meteo, LLM ou Hugging Face indisponibles entrainent un repli deterministe, jamais une page cassee.
- **Communication standard** : chaque agent publie une carte A2A et accepte des taches JSON-RPC, ce qui permet a l'orchestrateur de deleguer sans connaitre les routes internes.

2. Le produit : onze ecrans, huit agents

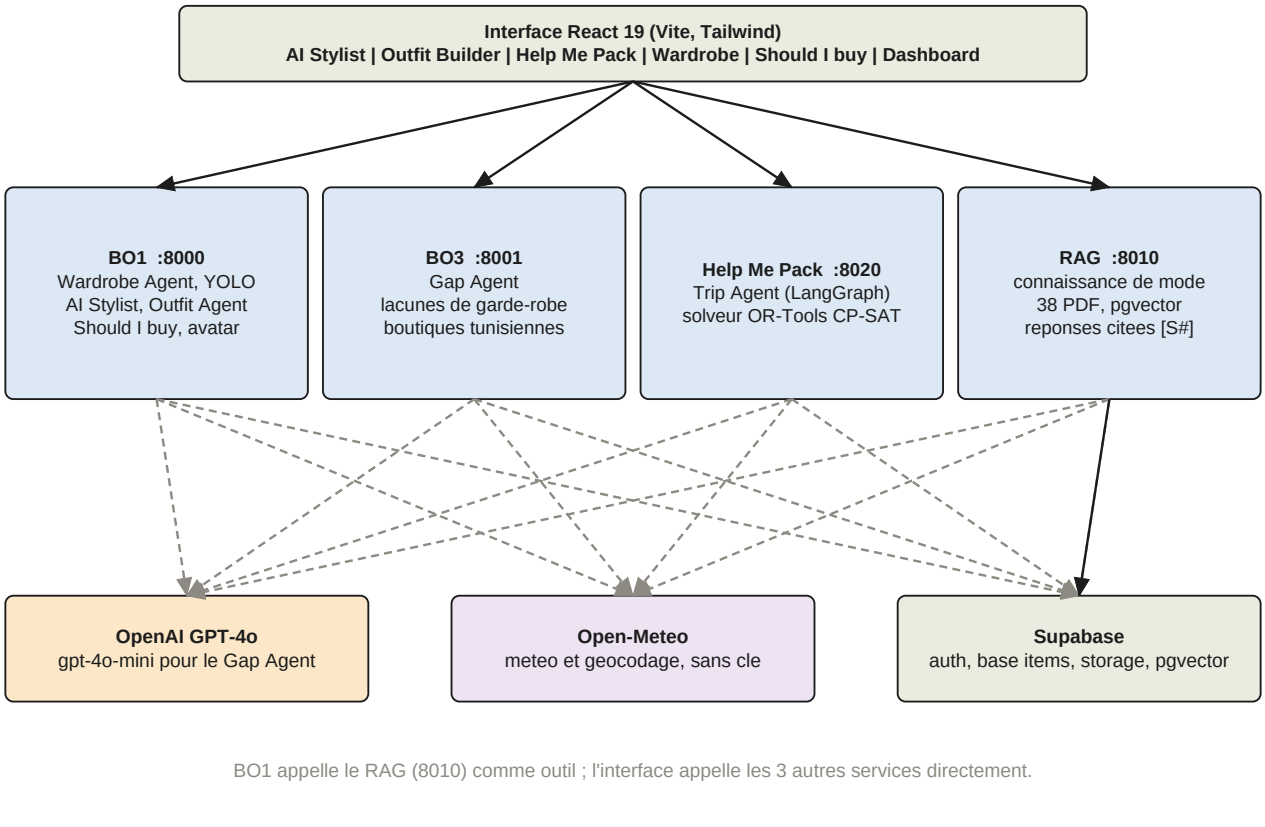
Chaque page de l'interface appelle une route ; la route appelle soit un **agent** (le LLM choisit des outils ou suit un graphe), soit un **moteur** (detecteur, moteur d'achat, comparaison locale).

Ecran	Route	Agent / moteur	Entree	Sortie
AI Stylist + voix	POST /api/agent/search	AI Stylist (LangGraph)	message, ville, pays, unite	conseil, meteo, tenue, raisons, sources
Wardrobe	POST /api/wardrobe/agent	Wardrobe Agent (LangGraph)	message, action	pieces, filtres, proposition
Outfit Builder	POST /api/outfits/agent	Outfit Builder (LangGraph)	occasion, saison, ville, avoid	3 tenues validees + sources
Help Me Pack	POST :8020/api/trips	Trip Agent + CP-SAT	message, session_id	valise, checklist, alternatives
Wardrobe Gaps	POST :8001/api/bo3/agent	Gap Agent + XGBoost	message, objectif	lacunes classees, pieces couvertes
Live scan	/api/items/quick-detect + check-duplicate	YOLO + Wardrobe Agent	flux camera	cadres, qualite, doublon
Should I buy this?	POST /api/shopping-assistant /evaluate	Qwen + 6 outils + GPT-4o	photo de l'article	verdict BUY / CONSIDER / SKIP
Find Similar	POST /api/items/analyze (all_items)	YOLO + comparaison locale	photo d'une tenue	owned / close / missing
Ajout, garde-robe, 3D	/api/items/analyze, /api/items	YOLO + catalogage vision	photo	fiche vetement, modele 3D
Dashboard	/api/items + :8001 gaps	Gap Agent + regles locales	aucune	statistiques, tenue du jour
Avatar, essayage	/api/avatar/*, tryon-outfit	WaveSpeed / fal.ai	photo, tenue	image generee, suivi de tache

Les agents ne filtrent plus la garde-robe eux-memes : ils la **demandent au Wardrobe Agent** en langage naturel, par appel direct dans BO1 ou par A2A pour le Trip Agent. Trois composants LLM ne sont pas des agents : le catalogage vision, le calcul des preuves d'achat et la generation de reponse du RAG.

3. Architecture globale

Quatre services FastAPI independants sont appeles par l'interface React 19. Supabase fournit l'authentification, la base de donnees et le stockage. OpenAI fournit les LLM, Open-Meteo la meteo reelle.



BO1 appelle le RAG (8010) comme outil ; l'interface appelle les 3 autres services directement.

Figure 1 : services, ports et dependances externes.

Service	Port	Stack	Role
BO1 (backend/app)	8000	FastAPI, LangGraph, YOLO	garde-robe, detecteur, AI Stylist, Outfit Agent, Similar Agent, Purchase Agent, avatar
BO3 (DressMe UI Blueprint)	8001	FastAPI, gpt-4o-mini, XGBoost	Gap Agent, produits de boutiques tunisiennes
Help Me Pack	8020	FastAPI, LangGraph, OR-Tools	Trip Agent, solveur CP-SAT, checklist
RAG (rag_backend)	8010	FastAPI, pgvector, MiniLM	connaissance de mode citee, reponses [S#]
Orchestrateur autonome	8070	FastAPI, LangGraph, Ollama	ancien orchestrateur, conserve comme alternative locale
Interface (Vite)	8443	React 19, Tailwind	application web, https de developpement
Ollama	11434	qwen2.5:3b	LLM local du Stylist BO4 et du routeur 8070

4. Donnees, authentification et memoire

Chaque requete est portee par un jeton Supabase. Une dependance FastAPI le verifie et expose un **user_id** ; toutes les requetes SQL sont filtrees par cet identifiant. Un utilisateur ne peut donc jamais lire ou modifier le placard d'un autre, y compris via une tache A2A.

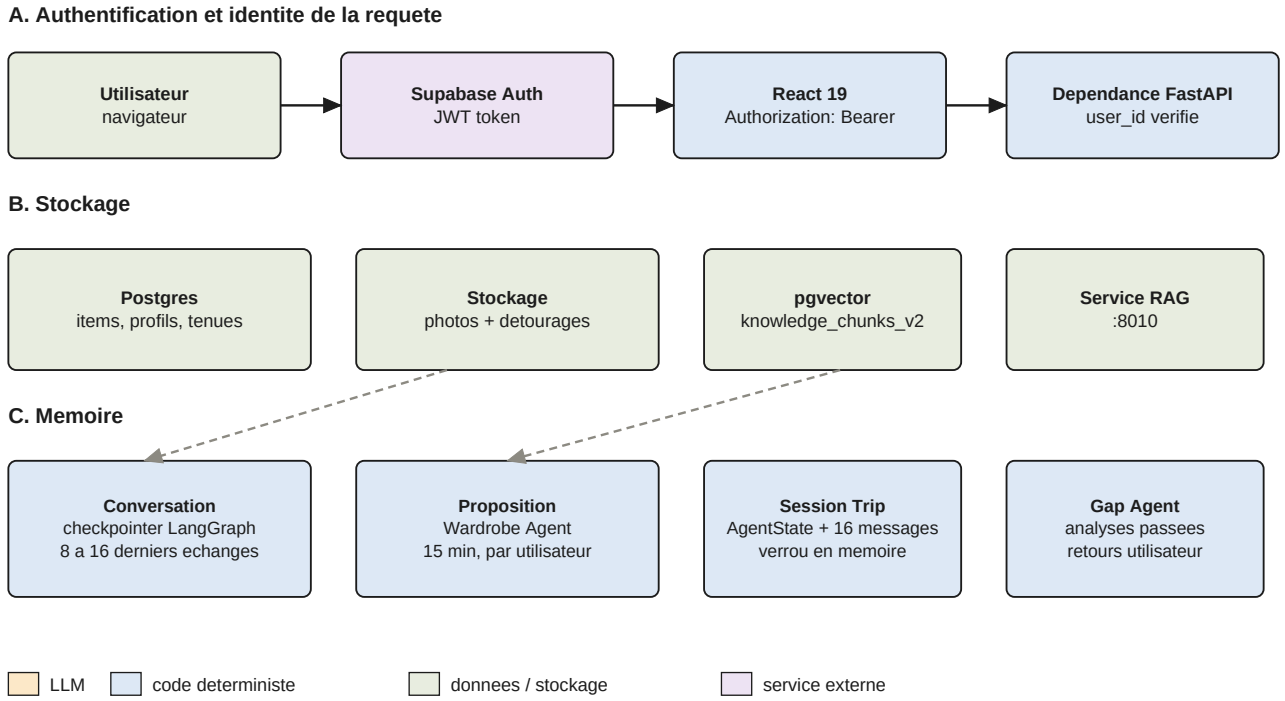


Figure 2 : identite de la requete, stockage et memoires.

Stockage

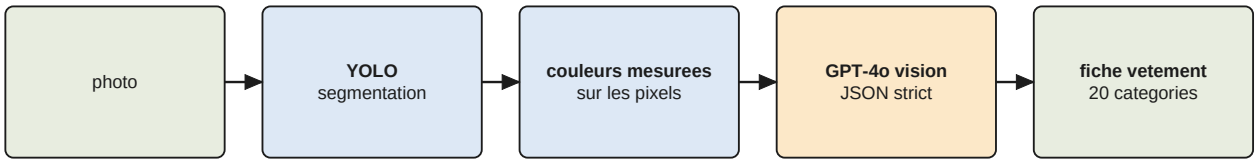
Store	Contenu	Utilise par
Postgres items	fiche vetement : categorie, couleur, matiere, saison, occasion, style, chaleur, image	tous les agents
Postgres profiles	ville, pays, unite, preferences, departement	AI Stylist, Outfit, Gap
Storage	photos originales et cutouts transparents	interface, essayage, 3D
pgvector knowledge_chunks_v2	14 574 passages + embeddings 384 d, index HNSW cosinus	service RAG
Outfits, feedback	tenues sauvegardees, retours sur les suggestions	Outfit Builder, Gap Agent

Memoires des agents

Memoire	Portee	Duree
Checkpointer LangGraph	un fil par utilisateur et par conversation ; 8 derniers echanges	persistante cote serveur
Wardrobe Agent	proposition de correction en attente (oui / non)	15 minutes, liee a l'utilisateur
Trip Agent	AgentState complet + 16 derniers messages	session en memoire, verrou par session_id
Gap Agent	analyses passees et retours utilisateur	base de donnees
RAG	historique de conversation	session du service

5. Pipeline vision : de la photo a la fiche

Le catalogage est un pipeline en deux temps. Le detecteur local fait le travail geometrique et colorimetrique ; le LLM vision fait le travail semantique.



D'abord le detecteur, ensuite le LLM : le modele affine mais n'invente jamais un vetement absent de la photo.

Sans cle API ou en cas d'erreur du modele : repli deterministe (fallback_enrich), la fiche reste utilisable.

Figure 3 : pipeline de catalogage d'un vetement.

- **YOLO** (`dressme_best.pt`) detecte et segmente le vetement sur la photo.
- Les **couleurs sont mesurees sur les pixels** du masque, puis ramenees a la palette la plus proche : le LLM ne devine pas la couleur.
- **GPT-4o vision** recoit l'image et un schema JSON strict : categorie, matiere, saison, occasion, style, formalite, niveau de chaleur, confiance. Il ne peut changer la categorie du detecteur que si l'image le montre clairement.
- Sans cle API ou en cas d'erreur, `fallback_enrich` applique des regles et la fiche reste utilisable.
- 20 classes : t-shirt, chemise, pull, veste, manteau, blouse, robe, jupe, pantalon, jean, short, combinaison, chaussures, bottes, sandales, sac, ceinture, chapeau, echarpe, accessoire.

6. LLM et modeles utilises

Le choix du modele depend de la tache : GPT-4o pour le raisonnement et la vision, gpt-4o-mini pour une tache courte et bon marche, Qwen + LoRA pour une decision specialisee, Ollama pour fonctionner sans cle API.

Composant	Modele	Temp.	Format	Limite	Role
Catalogage vision	GPT-4o	-	JSON strict	1 appel	categorie, couleur, matiere, saison, occasion
AI Stylist	GPT-4o	0,3	JSON	8 tours, 1 800 tokens	conseil, meteo, RAG, placard
Wardrobe Agent	GPT-4o	0,1	JSON	6 tours	recherche, comptage, doublons, correction
Similar Agent	GPT-4o	0,1	JSON	10 tours	statuts owned / close / missing
Outfit Builder (compose)	GPT-4o	0,6	JSON	2 passes, 2 200 tokens	3 tenues avec identifiants reels
Trip Agent	GPT-4o	0	outils + JSON	12 iterations	choix d'outils, valise CP-SAT
Gap Agent	gpt-4o-mini	0,2	Markdown	6 etapes	lacunes de garde-robe
Purchase Agent	Qwen2.5-3B + LoRA	0,1	JSON	6 tours	verdict BUY / CONSIDER / SKIP
Purchase (redaction)	GPT-4o	0,5	JSON 3 sections	-	reformule le verdict, ne decide pas
Purchase Redundancy	GPT-4.1 / Claude Opus 5.5	0,2	JSON	1 appel	redondance + CLIP ViT-B/32
Stylist BO4	Ollama qwen2.5:3b	0,2	JSON	9 noeuds	tenues locales, heuristique
Orchestrateur 8070	Ollama qwen2.5:3b	0	JSON / regles	1	routage d'intention
Recherche RAG	MiniLM + LLM	0,1	texte cite	top 5	reponse sourcee [S#]

7. Entrees et sorties par fonctionnalite

Toutes les routes sont protegees par le jeton Supabase. Les reponses sont du JSON ; l'interface affiche ensuite des zones dediees (meteo, tenues, raisons, sources, verdict).

Fonction	Entree	Sortie principale
AI Stylist	{message, city, country, unit}	{answer, weather, outfit[], why[], alternatives[], tips[], sources[], items[]}
Wardrobe	{message, action?, confirm?}	{reply, items[], filters, proposal{fields, item_id}}
Similar	{pieces[]} via A2A	{pieces[{status: owned|close|missing, match_item, missing_spec}]}
Outfit Builder	{occasion, season, city, avoid[]}	{outfits[{title, items[], weather_note, why[]}], missing[], sources[]}
Trip Agent	{message, session_id}	{message, state, capsule[], checklist[], alternatives[]}
Gap Agent	{message, goal}	{answer, gaps[], covered[], products[]}
Purchase Agent	{image} ou photo	{decision, score, reasons[], matching_item_ids[], decision_source}
Avatar	{photo} / {outfit_id}	{job_id, status, result_url}

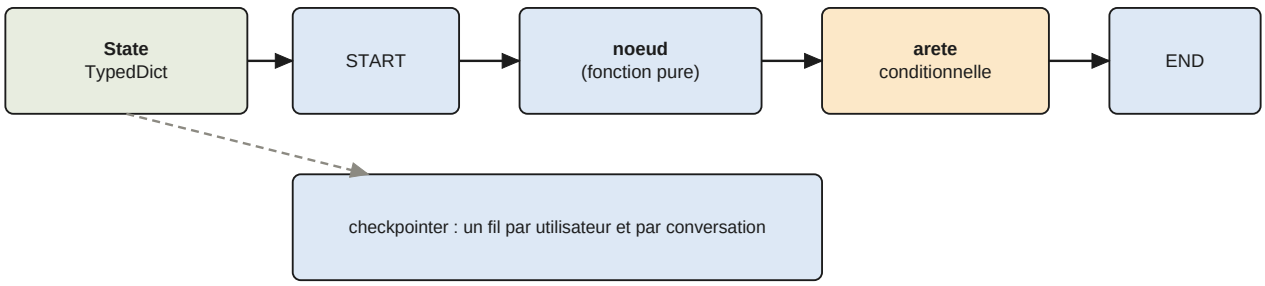
Exemple de reponse de l'AI Stylist

```
{
  "answer": "Wear the beige blazer with the white shirt and navy trousers.",
  "weather": "27 C and sunny in Tunis; light layers are enough.",
  "outfit": ["Beige blazer + White shirt + Navy trousers + Leather loafers"],
  "why": ["The blazer keeps the dinner look formal [S2].", "..."],
  "alternatives": ["The grey cardigan is too casual for a dinner."],
  "tips": ["Choose breathable cotton for a warm evening [S4]."]
}
```

8. LangGraph : comment fonctionne un agent

LangGraph modelise un agent comme une **machine a etats**. Un etat partage (un TypedDict) traverse des noeuds ; chaque noeud est une fonction Python qui lit l'etat et renvoie une mise a jour partielle. Les aretes conditionnelles choisissent le noeud suivant. Un checkpointer conserve l'etat entre deux appels.

A. Anatomie d'une machine a etats LangGraph



B. Boucle d'outils partagee (agent_kit/tool_loop.py)

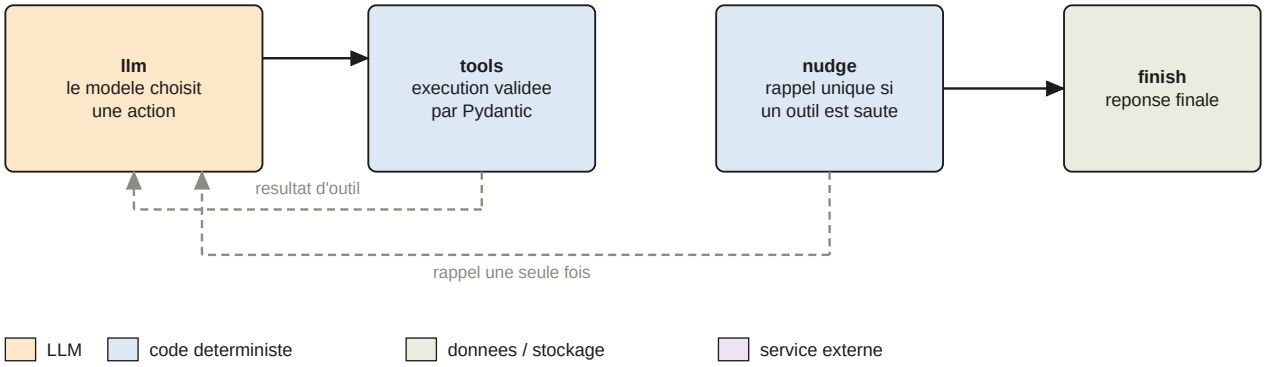


Figure 4 : anatomie d'un graphe et boucle d'outils commune.

Les quatre briques

Brique	Role dans DressMe
Etat partage	messages, items, meteo, conseils, brouillon de tenues, erreurs, identifiants de conversation
Noeud	une fonction pure : appeler un outil, construire le placard, composer, valider, rediger
Arete conditionnelle	le code decide du chemin (ex. gate -> apply / cancel / assist)
Checkpointer	memoire par utilisateur et par fil ; reprise d'une conversation apres un InputRequired

La boucle d'outils partagee

Le module `agent_kit/tool_loop.py` fournit la meme mecanique a tous les agents : le LLM choisit un outil, le code valide les arguments et execute l'outil, le resultat revient au LLM. Un noeud **nudge** envoie un rappel unique si le modele saute un outil obligatoire (meteo, RAG, placard). Un noeud **finish** arrete la boucle quand la reponse est exploitable.

Chaque agent reutilise ce squelette : un noeud **llm**, un noeud **tools**, une arete conditionnelle vers **finish** et une arete de retour **tools -> llm**. Seuls le modele, les outils et les garde-fous changent d'un agent a l'autre.

9. Les graphes, un par un

Chaque agent possede son graphe. Les noeuds sont nommes comme dans le code, ce qui permet de relier directement ce document aux fichiers sources.

9.1 AI Stylist : boucle d'outils

Modele GPT-4o, temperature 0,3, sortie JSON, 8 tours maximum. Trois outils : `search_wardrobe`, `get_weather`, `fashion_advice`.

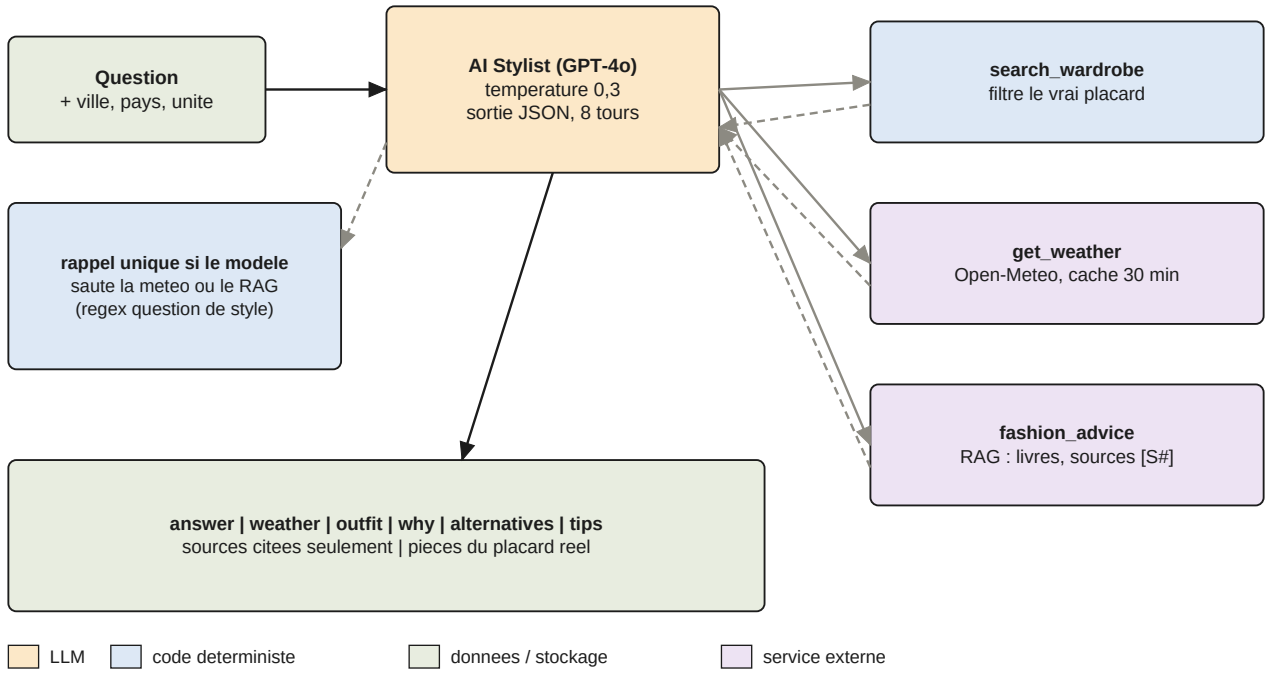


Figure 5 : boucle d'outils de l'AI Stylist.

- Le prompt impose d'appeler **search_wardrobe** avant de repondre et de ne parler que de pieces possedees.
- Toute question de style declenche **fashion_advice** ; toute question de meteo declenche **get_weather**.
- Un rappel unique est envoye si le modele oublie la meteo ou le RAG (detection par expression reguliere FR / EN).
- Seules les sources reellement citees sont renvoyees ; les pieces sont retrouvees par leur nom dans le placard.

9.2 Wardrobe Agent : graphe de confirmation

Le Wardrobe Agent possede la garde-robe. Son graphe commence par un noeud **gate** : si une correction attend une reponse, un oui mene a **apply**, un non a **cancel** ; sinon **assist** lance la boucle d'outils. Aucun outil LLM n'ecrit : `update_item` ne fait que proposer.

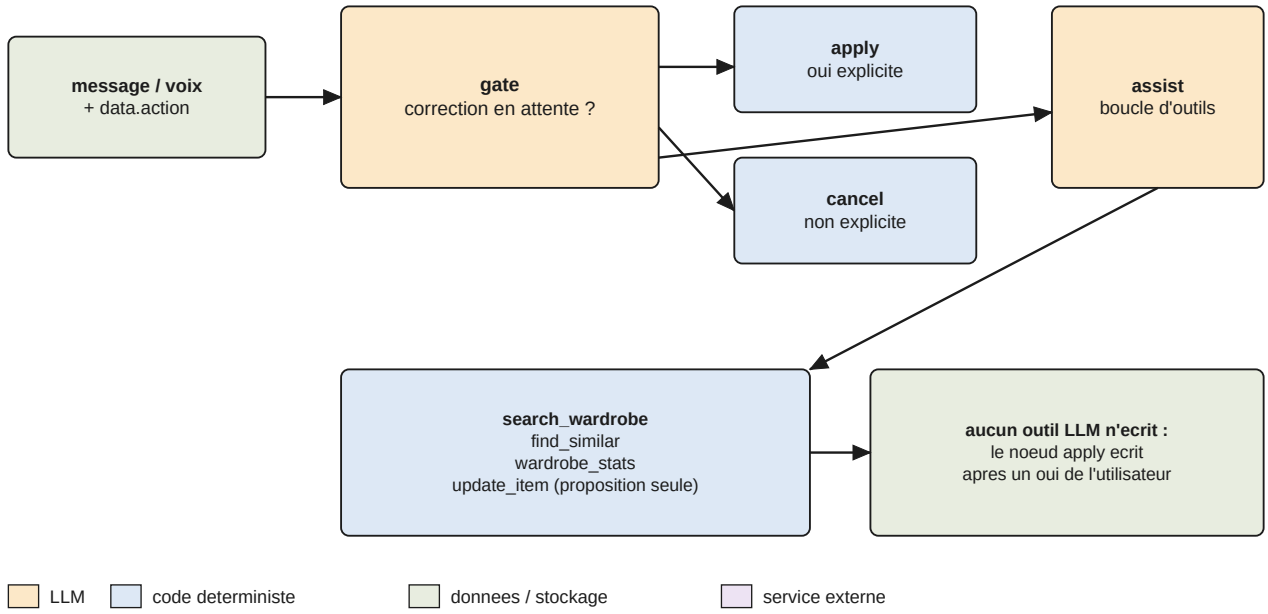


Figure 6 : gates et outils du Wardrobe Agent.

Outil	Role
search_wardrobe	filtre la garde-robe (categorie, couleur, saison, occasion, style, chaleur, texte)
find_similar	pieces proches avec les raisons (meme type, meme couleur, meme matiere)
wardrobe_stats	total, repartition, pieces a verifier
update_item	propose une correction, sans ecriture

Une correction n'est appliquee que par le noeud **apply**, apres un oui explicite. La proposition expire apres 15 minutes et reste liee a l'utilisateur et a la conversation. Le meme agent expose des actions structurees sans LLM (list, search, check-duplicate) utilisees par les autres agents et le Live scan.

9.3 Similar Agent : owned, close, missing

A partir d'une photo de tenue, le detecteur decoupe les pieces ; l'agent demande au Wardrobe Agent (A2A) les pieces proches, propose un statut par piece, puis le **code verifie** chaque identifiant cite.

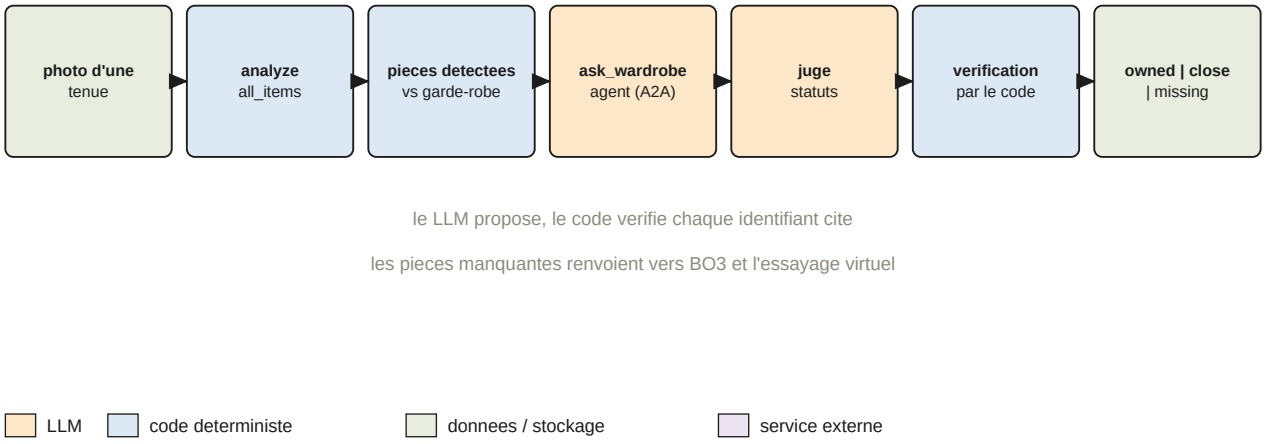


Figure 7 : chaine Similar Agent.

- **owned** : la piece exacte est dans le placard ; **close** : meme categorie et couleur proche ; **missing** : rien ne correspond.
- Le code valide les statuts et rejette tout identifiant absent du placard.
- Une piece manquante renvoie vers BO3 (photos de boutiques tunisiennes) et vers l'essayage virtuel.

9.4 Outfit Builder : graphe metier a cinq noeuds

Le LLM n'intervient que dans **compose**. Les quatre autres noeuds sont du code deterministe, ce qui garantit que la meteo, le placard et la validation ne dependent pas du modele.

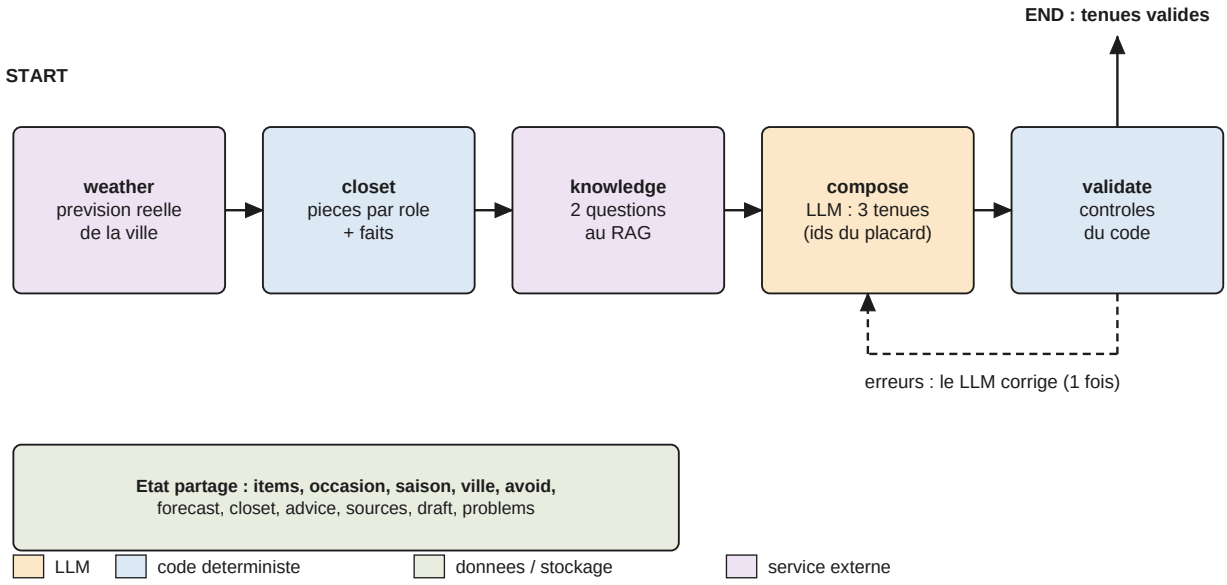


Figure 8 : graphe de l'Outfit Builder.

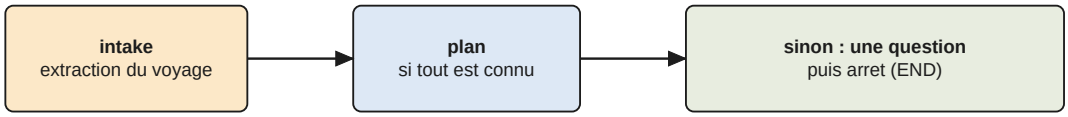
Noeud	Entree	Sortie	Type
weather	ville, pays	forecast ou None	Open-Meteo
closet	items	pieces classees par role (13 champs)	code
knowledge	occasion, saison, forecast	advice[] + sources	RAG, 2 questions
compose	brief complet	outfits[] {title, items[], weather_note, why[]}, missing[]	GPT-4o, JSON, temp. 0,6
validate	outfits[] + placard	tenues valides ou erreurs (retour a compose une fois)	code

Regles de validation : identifiants reels ; robe seule ou haut + bas ; chaussures obligatoires ; jamais deux pieces du meme role ; veste si froid ou pluie. En cas d'echec, le LLM est relance **une seule fois** avec la liste exacte des erreurs.

9.5 Trip Agent : deux graphes et dix outils

Le Trip Agent combine une conversation (graphe 1) et un workflow deterministe (graphe 2). Le LLM n'invente ni destination, ni dates, ni meteo, ni resultat du solveur ; il choisit l'outil suivant.

Graphe 1 : conversation (LangGraph)



Graphe 2 : workflow deterministe (chaque noeud s'arrete si l'etape precedente echoue)



Boucle d'outils GPT-4o (temperature 0, 12 iterations max) : 10 outils valides par Pydantic

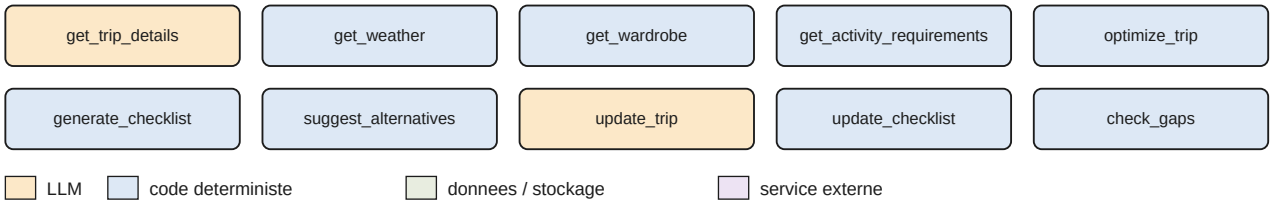


Figure 9 : les deux graphes du Trip Agent et ses dix outils.

Outil	Role	Ressource
get_trip_details	extrait, fusionne et valide les champs du voyage	GPT-4o JSON + regex
get_weather	prevision jour par jour pour la destination	Open-Meteo
get_wardrobe	charge la garde-robe et la couverture des roles	CSV + pandas
get_activity_requirements	traduit activites et meteo en exigences	regles
optimize_trip	choisit la valise (moins de pieces, plus combinables)	OR-Tools CP-SAT
generate_checklist	checklist depuis la capsule	Pydantic
suggest_alternatives	diagnostique INFEASIBLE sans relacher les contraintes	CP-SAT
update_trip	modifie le voyage et invalide l'etat obsolete	etat de l'agent
update_checklist	coche une piece sans relancer le solveur	etat de l'agent
check_gaps	appelle le Gap Agent par A2A (optionnel)	fonction injectee

9.6 Gap Agent : lacunes et classement XGBoost

Modele gpt-4o-mini, temperature 0,2, 6 etapes maximum. Les **regles deterministes** decident si une lacune existe ; XGBoost ne fait que l'ordonner. Le prompt precise que le classeur est entraine sur des donnees synthetiques tant qu'il n'y a pas de retours reels.

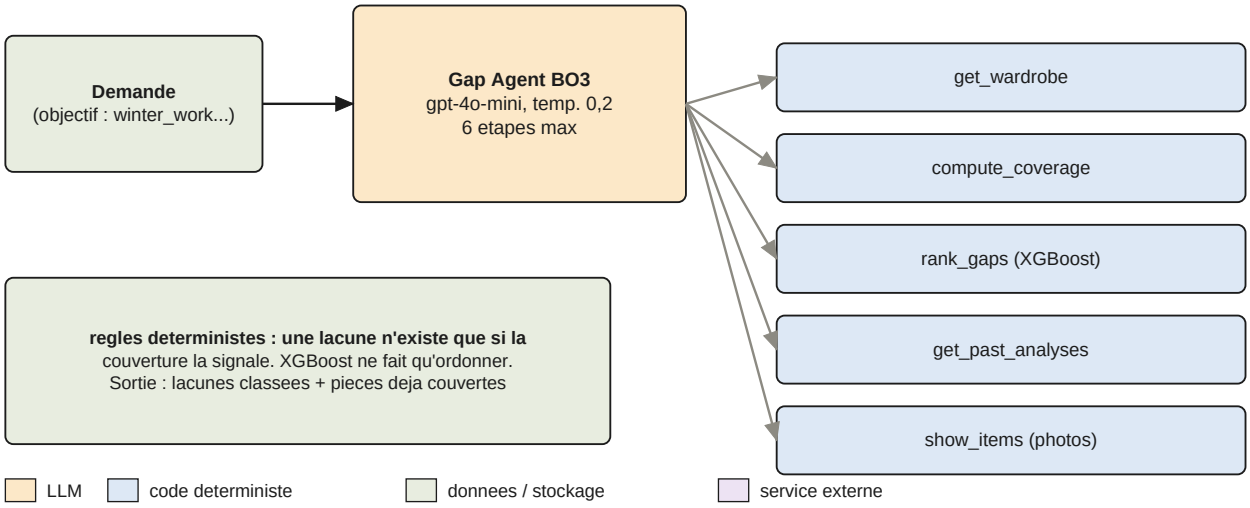


Figure 10 : Gap Agent et ses cinq outils.

- **compute_coverage** compare l'objectif choisi aux pieces possedees, par saison et par exigence.
- **rank_gaps** score chaque manque (deficit, frequence, saison, gain de tenues, preference, redondance FashionCLIP) puis ordonne avec XGBoost.
- **get_past_analyses** donne une memoire a long terme ; les retours alimentent `/api/bo3/retrain`.
- Jamais de piece d'un autre departement ; jamais de genre devine. Reponse dans la langue de l'utilisateur.

9.7 Purchase Agent : Qwen decide, GPT redige

Le decideur est **Qwen2.5-3B-Instruct + adaptateur LoRA** (`meriemgharbi123/dressme_qwen3b_lora`), charge depuis Hugging Face. Il choisit lui-meme six outils dans une boucle LangGraph ; chaque outil est calcule par du code sur la vraie garde-robe, et ses resultats font foi.

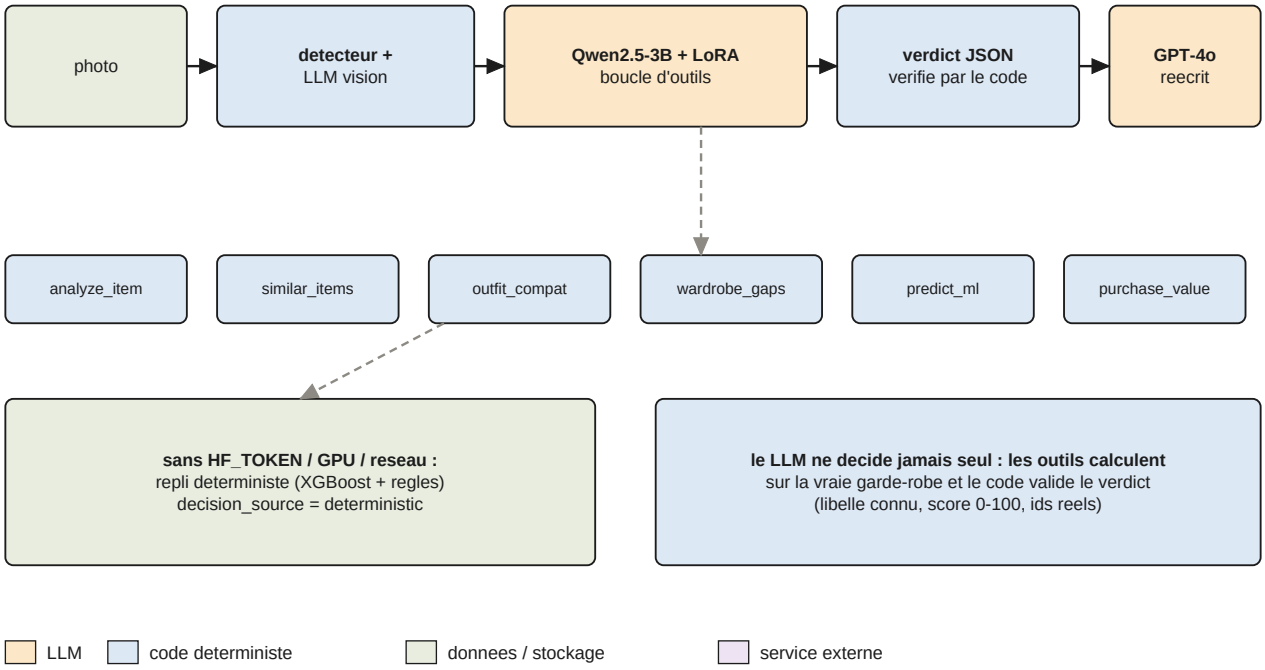


Figure 11 : chaine de decision du Purchase Agent.

Outil	Ce qu'il calcule
analyze_item	categorie, couleur et attributs de l'article photographie
find_similar_items	pieces proches deja possedees
check_outfit_compatibility	tenues possibles avec l'article
analyze_wardrobe_gaps	lacune comblee ou non
predict_purchase_ml	score XGBoost de valeur d'achat
calculate_purchase_value	rapport cout / usage estime

Qwen termine par un verdict JSON {decision, score, matching_item_ids, reasons}. Le code le verifie : decision connue, score entre 0 et 100, identifiants reellement presents, raisons courtes. GPT-4o recoit ce JSON et le reecrit en trois sections, sans pouvoir changer la decision. Repli deterministe si Hugging Face, GPU ou reseau indisponibles ; `decision_source` indique la source.

9.8 Stylist BO4 : graphe local a neuf noeuds

BO4 est une pile separee, non branchee a l'interface React. Elle montre une variante locale, sans cle API : Ollama qwen2.5:3b et un moteur de tenues heuristique.

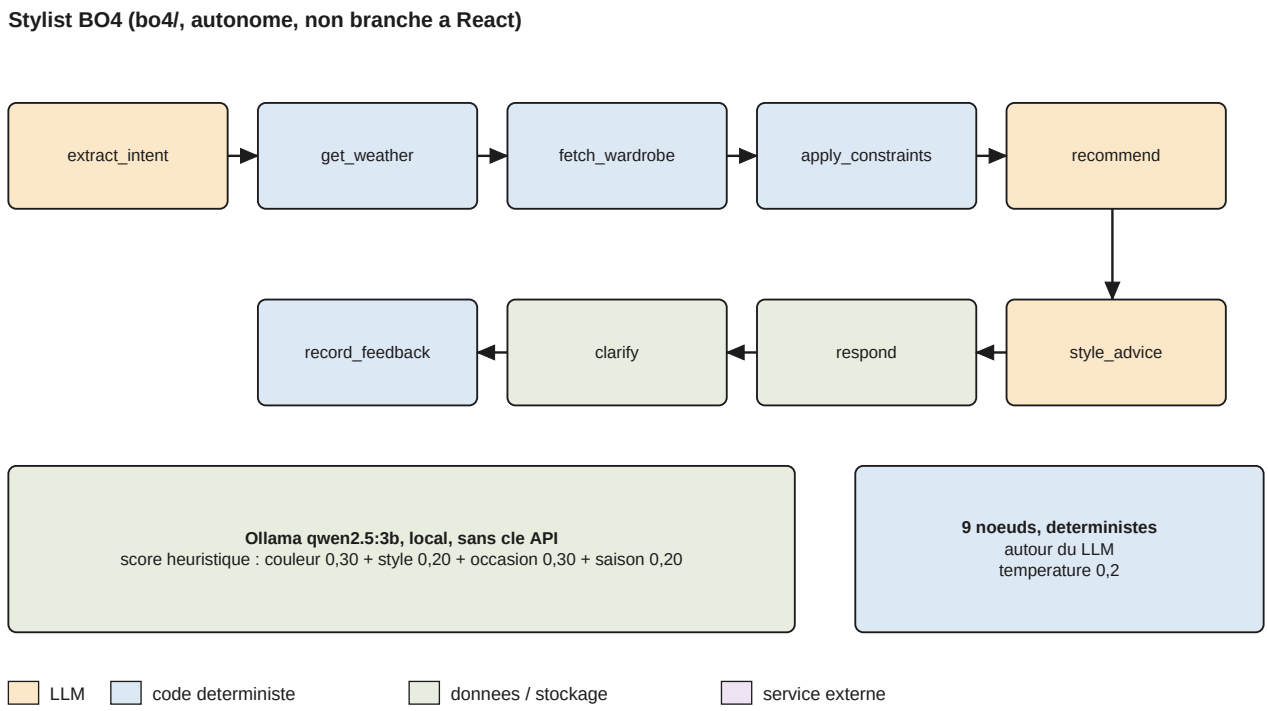


Figure 12 : graphe du Stylist BO4.

Les neuf noeuds : extract_intent, get_weather, fetch_wardrobe, apply_constraints, recommend, style_advice, respond, clarify, record_feedback. Le score heuristique combine couleur (0,30), style (0,20), occasion (0,30) et saison (0,20).

10. Communication entre agents : A2A

Chaque agent expose une **carte d'agent** (identite, competences, authentification) et un point d'entree **JSON-RPC** conforme au protocole Agent2Agent 0.3.0. N'importe quel client peut donc decouvrir un agent et lui confier une tache par HTTP, sans connaitre ses routes internes.

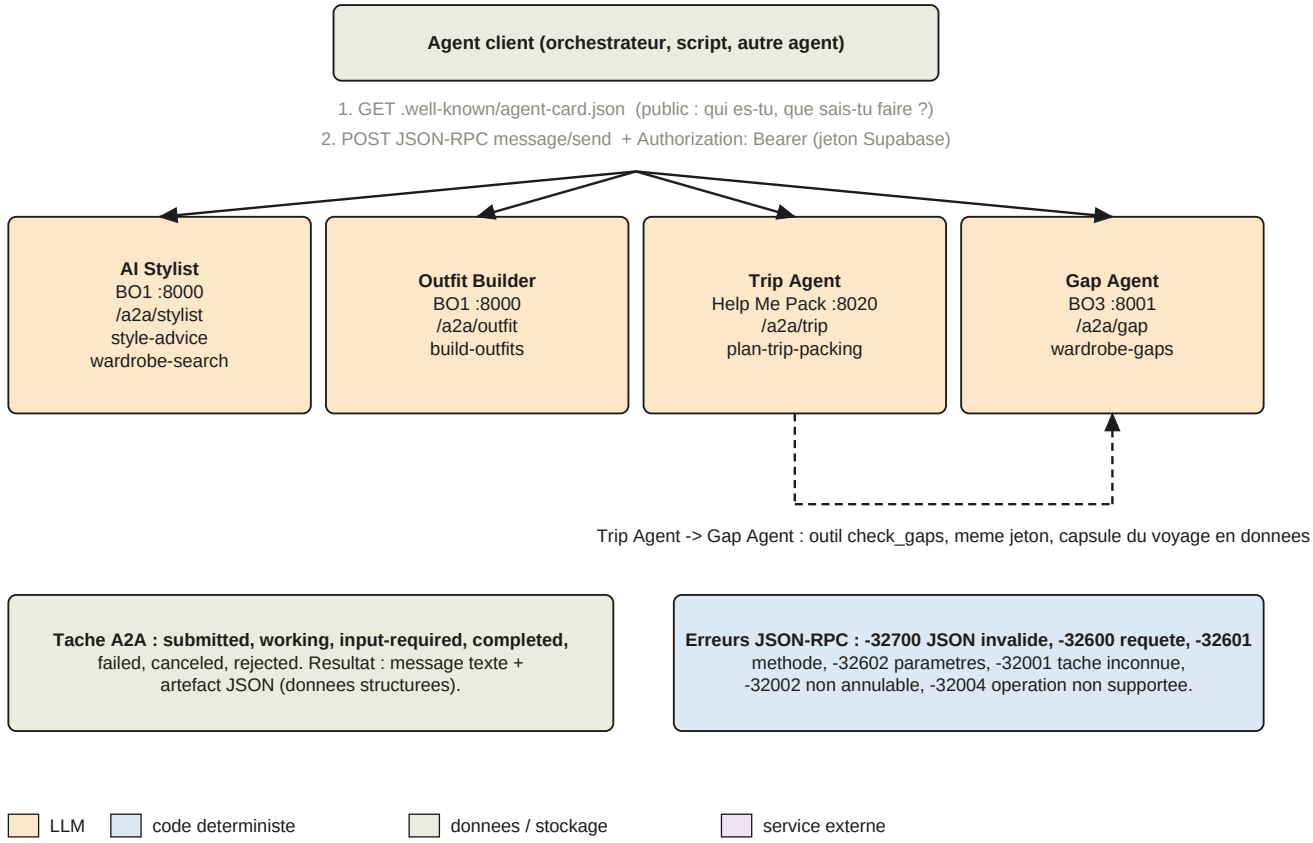


Figure 13 : decouverte par carte, envoi d'une tache, agent vers agent.

Agent	Carte	Point d'entree	Competence
AI Stylist	/a2a/stylist/.well-known/agent-card.json	POST /a2a/stylist (8000)	style-advice, wardrobe-search
Outfit Builder	/a2a/outfit/.well-known/agent-card.json	POST /a2a/outfit (8000)	build-outfits
Wardrobe Agent	/a2a/wardrobe/.well-known/agent-card.json	POST /a2a/wardrobe (8000)	search, list, correction, stats, doublons
Similar Agent	/a2a/similar/.well-known/agent-card.json	POST /a2a/similar (8000)	find-similar
Trip Agent	/a2a/trip/.well-known/agent-card.json	POST /a2a/trip (8020)	plan-trip-packing
Gap Agent	/a2a/gap/.well-known/agent-card.json	POST /a2a/gap (8001)	wardrobe-gaps

Methodes et etats

- **message/send** (reponse synchrone), **tasks/get**, **tasks/cancel**.
- Etats d'une tache : submitted, working, input-required, completed, failed, canceled, rejected.
- Erreurs JSON-RPC : -32700 analyse, -32600 requete, -32601 methode, -32602 parametres, -32001 tache inconnue, -32002 non annulable, -32004 operation non supportee.
- La carte est publique ; chaque tache exige le jeton Supabase et ne travaille que sur la garde-robe de cet utilisateur. Une tache n'est lisible que par son createur.
- **input-required** : le Trip Agent demande une precision ; le client repond avec le meme `contextId` (identifiant de session de voyage).

- Non implemente et declare tel quel : streaming (message/stream) et notifications push.

Exemple de tache envoyee au Wardrobe Agent :

```
POST /a2a/wardrobe
Authorization: Bearer <supabase_jwt>

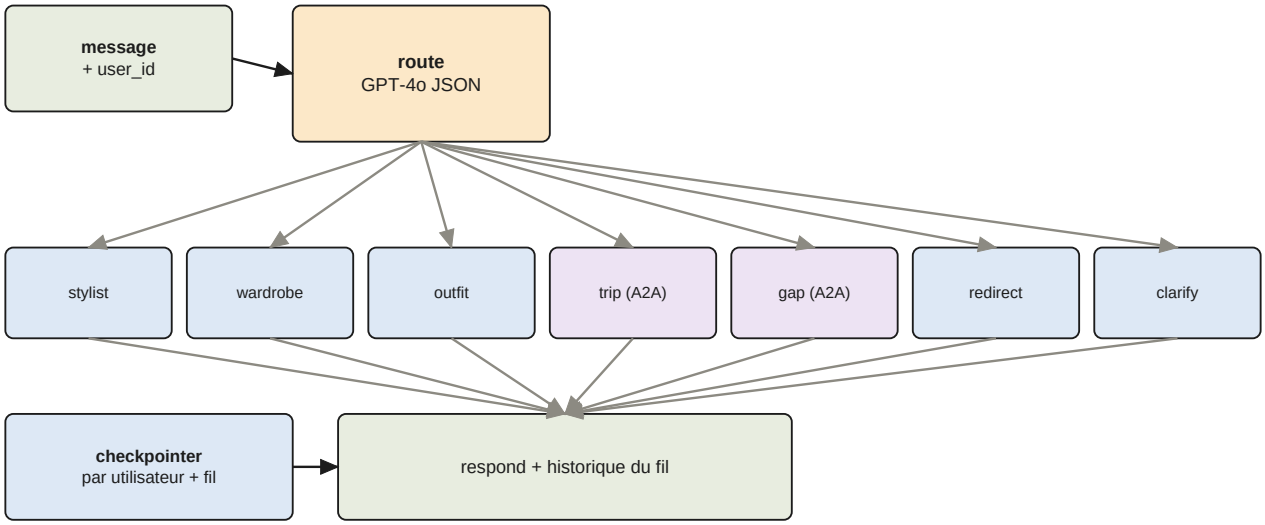
{"jsonrpc": "2.0", "id": "42", "method": "message/send",
 "params": {"message": {"role": "user", "parts": [{"kind": "text", "text": "list my jackets"}]},
            "metadata": {"action": "list"}}}
```

Agent vers agent : l'outil **check_gaps** du Trip Agent envoie la capsule du voyage au Gap Agent (A2A, meme jeton). Si le Gap Agent est arrete, l'outil renvoie une erreur propre et le voyage continue. L'adresse se regle avec `GAP_A2A_URL`.

11. Orchestrateurs

L'orchestrateur BO1 est le point d'entree conversationnel de l'application : `POST /api/assistant/chat`. Il choisit l'agent, lui delegue la demande et garde le fil. Les autres ecrans continuent d'appeler leur agent directement.

A. Orchestrateur BO1 : POST /api/assistant/chat



B. Orchestrateur autonome (orchestrator/, port 8070)

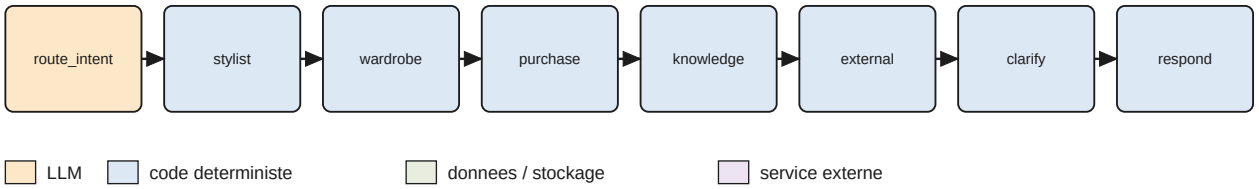


Figure 14 : orchestrateur BO1 et orchestrateur autonome 8070.

Noeud BO1	Role
route	GPT-4o en JSON, liste fermee d'agents ; regles FR / EN si le LLM est indisponible ; si un agent attend une reponse, la suite va au meme agent
stylist / wardrobe / outfit	appel direct dans BO1
trip / gap	appel A2A avec le jeton de l'utilisateur (8020 et 8001)
redirect	Should I buy et Find Similar ont besoin d'une photo : ouvre la bonne page
clarify	demande une precision si la demande est trop vague
respond	ajoute le tour a l'historique (8 derniers echanges, un fil par utilisateur)

Exemple de conversation suivie

- "Prepare ma valise pour Paris" : routage vers le Trip Agent, tache **input-required** ("quelles dates ?").
- "Du 3 au 7 juin, 8 kg" : aucun mot-cle d'agent, mais la question est restee ouverte ; la reponse repart au Trip Agent avec le meme contextId.
- Une correction du Wardrobe Agent n'est continuee que par un oui ou un non ; toute autre phrase est une nouvelle demande.
- Chaque reponse indique l'agent qui a repondu ("Answered by ...") ; les tenues s'affichent avec leurs photos et un bouton ouvre la page demandee.

Orchestrateur autonome 8070

Le graphe `orchestrator/` (port 8070) est conserve comme alternative locale : **route_intent** (Ollama qwen2.5:3b puis regles), branches stylist, wardrobe, purchase, knowledge, external, clarify, puis respond. Il n'est pas branche a l'interface React actuelle.

12. Le RAG : connaissance de mode citee

Le RAG repond a des questions de style a partir de 38 livres de mode. Il ne connait pas le placard : c'est l'agent qui fait le lien entre la regle du livre et les pieces reelles.

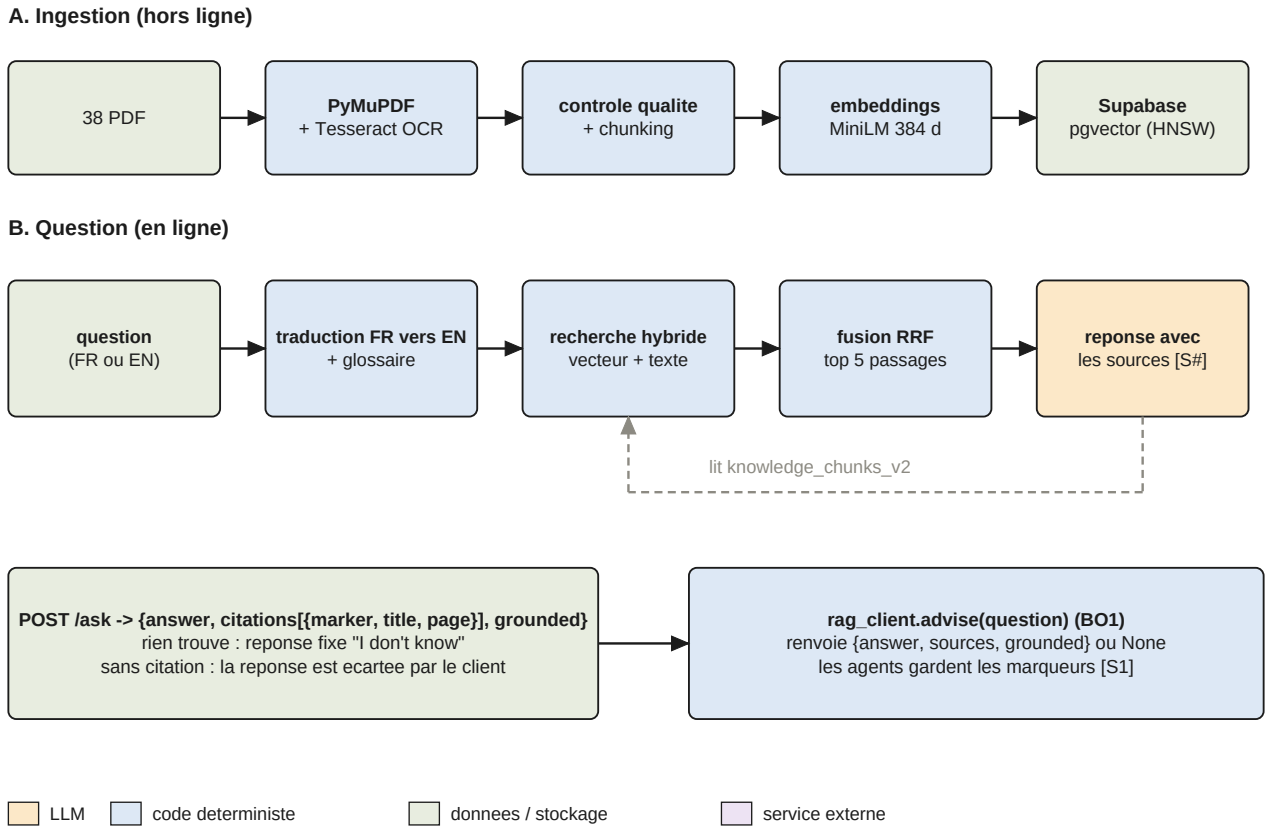


Figure 15 : ingestion hors ligne et interrogation en ligne.

Element	Detail
Corpus	38 PDF : couleurs, tissus, codes vestimentaires, capsules, accessoires
Ingestion	PyMuPDF, OCR Tesseract pour les livres scannes, controle qualite, chunking
Index	knowledge_chunks_v2, Postgres + pgvector, HNSW, cosinus, 384 dimensions
Embeddings	sentence-transformers all-MiniLM-L6-v2
Recherche	hybride : vecteurs + plein texte, fusion RRF, top 5 passages
Question	francais traduit en anglais avec glossaire ; anglais inchange
Generation	repond uniquement avec les sources numerotees ; sinon "I don't know"
API	POST /ask, /ask/stream (SSE), cle d'API, limitation de debit, historique

Qualite mesuree

Metrique	Valeur	Ce qu'elle mesure
Hit@5	91,67 %	le bon passage est dans les cinq premiers
MRR	85,42 %	rang moyen du premier bon passage
Precision du contexte	71,67 %	part des passages recuperes qui sont pertinents
Chunks indexes	14 574	taille de l'index pgvector
Documents	38	livres de mode traites

Integration : `rag_client.advise(question)` renvoie {answer, sources, grounded} ou None. L'AI Stylist l'expose comme outil **fashion_advice** ; l'Outfit Builder l'appelle dans le noeud **knowledge**. Les marqueurs [S#] sont conserves puis affiches en badges (livre et page).

13. Algorithmes deterministes et apprentissage

Le projet combine plusieurs familles d'algorithmes. Les LLM ne remplacent jamais un calcul exact quand celui-ci est possible.

Algorithme	Utilise par	Role
YOLO (dressme_best.pt)	catalogage, live scan, similar, achat	detection et segmentation des vetements
OR-Tools CP-SAT	Trip Agent	choix optimal de la valise sous contraintes dures
XGBoost	Gap Agent, Purchase Agent	classement des lacunes et score d'achat
Heuristique de tenues	Outfit Builder (repli), BO4	couleur 0,30 + style 0,20 + occasion 0,30 + saison 0,20
FashionCLIP	Gap Agent, redondance d'achat	similarite visuelle entre pieces
CLIP ViT-B/32	Purchase Redundancy	redondance semantique d'un article
Recherche hybride + RRF	RAG	fusion des classements vecteur et texte
Regles metier	Wardrobe, Outfit, Trip, Gap	validation, contraintes, garde-fous

Le solveur de valise

- Variables : une par piece du placard, binaire (prise ou non).
- Contraintes dures : au moins une tenue par jour et par activite, exigences meteo, formalite, limite de poids, pas de doublon de role.
- Objectif : minimiser le poids puis le nombre de pieces, en maximisant les combinaisons possibles.
- Si le probleme est **INFEASIBLE**, `suggest_alternatives` explique la contrainte bloquante sans jamais la relacher silencieusement.

14. Avatar et essayage virtuel

L'avatar est genere a partir d'une photo en pied ; l'essayage applique ensuite une tenue ou un vetement sur cet avatar. Le fournisseur est configurable avec `AVATAR_PROVIDER`.

- **Avatar 3D** : Hunyuan3D v3 via WaveSpeed (defaut) ou fal.ai.
- **Essayage d'une tenue complete** : `/api/avatar/tryon-outfit` (Bria virtual try-on).
- **Essayage d'un vetement de boutique** : `/api/avatar/tryon-garment`.
- Les generations sont des **taches d'arriere-plan** : la route renvoie un job_id, l'interface suit le statut puis affiche le resultat.
- Une piece manquante detectee par Find Similar peut etre essayee avant l'achat.
- Aucune image d'avatar n'est envoyee a un agent LLM : seuls les fournisseurs d'images la voient.

15. Securite, garde-fous et tests

Theme	Mesure
Ancrage	identifiants du placard uniquement ; outils en lecture ; sources numerotees ; meteo reelle
Validation par code	Outfit (regles de composition), Trip (Pydantic + solveur), Purchase (verdict), Similar (identifiants)
Degradation douce	RAG, meteo, LLM, HF indisponibles : reponse partielle ou repli par regles
Cloisonnement	jeton Supabase verifie sur chaque route ; requetes filtrees par user_id ; taches A2A privees
Secrets	cles API uniquement cote serveur ; jamais dans le frontend ni dans un prompt
Honnetete	le Gap Agent dit que XGBoost est entraine sur des donnees synthetiques ; le RAG repond "I don't know" plutot que d'inventer
Erreurs A2A	tache failed avec message neutre, aucun detail interne
Limitation de debit	service RAG et routes sensibles

Tests

Suite	Nombre	Remarque
Orchestrateur	101	routage, memoire, appels A2A simules
Backend BO1	85	faux LLM, faux RAG, fausse meteo
Frontend	30	composants et parcours
BO4	1 095 (25 ignores)	graphe local et heuristiques
Moteur d'achat, Trip Agent	tests dedies	outils, solveur, repli deterministe
Bout en bout	essais reels	GPT-4o, RAG, Open-Meteo

16. Limites assumees

- Le RAG ne connait pas le placard ; il ne contient que des livres de mode.
- Le classeur XGBoost du Gap Agent est entraine sur des donnees synthetiques jusqu'a l'arrivee de retours reels.
- L'AI Stylist n'a pas de memoire entre deux messages ; la memoire du Trip Agent est en memoire serveur et disparait au redemarrage.
- Le Stylist BO4 et l'orchestrateur 8070 ne sont pas branches a l'interface React.
- Le Purchase Agent depend de Hugging Face : sans HF_TOKEN il n'est pas actif ; Qwen2.5-3B en local demande assez de memoire et de GPU.
- La meteo couvre un seul jour a la fois (jusqu'a 15 jours) pour une seule ville.
- A2A est une implementation legere : JSON-RPC et cartes d'agent, sans streaming ni notifications push.
- Pistes : memoire de preferences par utilisateur, journal des tenues portees, orchestrateur 100 % A2A, migration de l'AI Stylist vers un graphe explicite.

17. Lancement et scenario de demonstration

Etape	Commande / variable	Port
BO1	.\start_bo1.ps1	8000
BO3	.\start_bo3.ps1	8001
Help Me Pack	.\start_packing.ps1	8020
RAG	.\start_rag.ps1	8010
Interface	pnpm dev	8443
Orchestrateur autonome	uvicorn orchestrator.main:app --port 8070	8070
Ollama	ollama serve	11434

Variables d'environnement utiles

Variable	Role
OPENAI_API_KEY	GPT-4o, gpt-4o-mini, vision
SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY	auth, base, stockage
HF_TOKEN / QWEN_BACKEND / HF_ENDPOINT_URL	Purchase Agent (local, hf_api, auto, off)
GAP_A2A_URL	adresse du Gap Agent pour l'outil check_gaps (vide pour desactiver)
RAG_API_URL / RAG_API_KEY	client RAG des agents
AVATAR_PROVIDER	wavespeed ou fal
OLLAMA_HOST	LLM local BO4 et orchestrateur 8070

Scenario de demonstration (10 minutes)

1. Ajouter trois vetements par photo : montrer la fiche generee et la couleur mesuree.
2. Ouvrir Wardrobe et demander "montre-moi mes vestes pour le travail" : le LLM choisit les filtres.
3. AI Stylist : "que mettre demain soir pour un diner a Tunis ?" : meteo, tenue, raisons, sources.
4. Outfit Builder : generer trois tenues, puis Shuffle pour en obtenir d'autres.
5. Help Me Pack : "valise pour Paris du 3 au 7 juin, 8 kg" : question de precision, puis checklist.
6. Wardrobe Gaps : objectif "winter work" : lacunes classees et produits de boutiques tunisiennes.
7. Should I buy this? : photographier un article et montrer le verdict, les preuves et le repli.
8. Dashboard : statistiques, tenue du jour et notifications.
9. Terminer par une tache A2A : `python scripts/a2a_discover.py --ask`.

18. Carte du depot

Chemin	Contenu
backend/app	BO1 : routes, agents LangGraph, detecteur, catalogage, avatar
DressMe UI Blueprint	BO3 et frontend React
packing_backend	Trip Agent, solveur CP-SAT, checklist
rag_backend	ingestion, index pgvector, service /ask
orchestrator	orchestrateur autonome 8070 (graphe, services.yaml)
a2a_protocol	serveur et client A2A, cartes d'agent
agent_kit	boucle d'outils LangGraph partagee
bo4	Stylist local a neuf noeuds (Ollama)
smart_purchasing	Purchase Agent Qwen + LoRA et outils d'achat
purchase-redundancy-ai	service de redondance GPT-4.1 / Claude + CLIP
scripts	lancement, decouverte A2A, maintenance
docs	guides techniques et PDF de presentation

19. Glossaire

Terme	Definition
Agent	composant qui combine un LLM, des outils et une politique de decision
LangGraph	bibliotheque de graphes d'etats pour agents, avec noeuds, aretes et checkpointer
A2A	protocole Agent2Agent : carte d'agent + taches JSON-RPC
RAG	retrieval-augmented generation : repondre a partir de documents recuperes et cites
Chunk	passage de texte indexe dans pgvector
RRF	Reciprocal Rank Fusion, fusion de plusieurs classements
Ancrage (grounding)	obligation de fonder chaque affirmation sur une donnee reelle
Garde-fou	regle de code ou de prompt qui empeche une sortie inventee
Checkpointer	mecanisme qui sauvegarde l'etat d'un graphe entre deux appels
Repli (fallback)	chemin deterministe quand un service externe est indisponible
CP-SAT	solveur de programmation par contraintes d'OR-Tools
LoRA	adaptateur leger qui specialise un modele pre-entraine